IT CAREER HUB

Django

Agile projects: Part 3



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- Камера должна быть включена на протяжении всего занятия
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Вести себя уважительно и этично по отношению к остальным участникам занятия
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

IT CAREER HUB

УЧЕБНЫЙ ПРОЕКТ

Задание 1

Получить список всех файлов проекта

1. Обновите существующее отображение **ProjectFileListGenericView**, дописав к нему метод **get_serializer_class.**
2. Добавьте в этот же класс метод **get_queryset()**, добавив возможность получения всех файлов проекта с поиском по конкретному проекту (как фильтр использовать имя проекта).
3. Добавьте в этот же класс метод **get()** на получение списка всех файлов проекта.
4. Протестируйте получение файлов проекта методом **GET.**
5. Закомментируйте все изменения, сделайте запрос на слияние.

Задание 1

Решение

1. Добавление необходимых методов (apps/projects/views/project_file_views.py):

```python
class ProjectFileListGenericView(ListCreateAPIView):
    def get_serializer_class(self):
        if self.request.method == 'GET':
            return AllProjectFilesSerializer
        return CreateProjectFileSerializer

    def get_queryset(self):
        project_name = self.request.query_params.get('project')

        if project_name:
            project_file = ProjectFile.objects.filter(
                project__name=project_name
            )
            return project_file

        return ProjectFile.objects.all()
```

1

```python
def list(self, request: Request, *args, **kwargs) -> Response:
    project_files = self.get_queryset()

    if not project_files.exists():
        return Response(
            data=[],
            status=status.HTTP_204_NO_CONTENT
        )

    serializer = self.get_serializer(project_files, many=True)

    return Response(
        serializer.data,
        status=status.HTTP_200_OK
    )
```

2

Задание 1

Решение

2. Запуск сервера, проверка **GET, POST** запросов.

3. Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 2

Получение файла, возможность его удаления

1. Создайте сериализатор **ProjectFileDetailSerializer** для получения подробной информации о файле:
    - **name**
    - **created_at**
    - **project (вложенный сериализатор для отображения названия проекта)**
2. Напишите классовое отображение **ProjectFileDetailGenericView** для получения информации о конкретном файле и проекте, к которому он привязан, а также возможности удаления этого файла, как из базы данных, так и из проекта.
3. Зарегистрируйте новый эндпоинт, проверьте как отрабатывают запросы **GET**, **DELETE.**
4. Зафиксируйте все изменения, сделайте запрос на слияние.

Задание 2

Решение

1. Создание сериализатора (apps/projects/serializers/project_file_serializer.py)**:**

```
class ProjectFileDetailSerializer(serializers.ModelSerializer):
   project = ProjectShortInfoSerializer(many=True)

   class Meta:
       model = ProjectFile
       exclude = ('file_path',)
```

Задание 2

Решение

2. Создание отображения ProjectFileDetailGenericView (apps/projects/views/project_file_views.py):

```python
class ProjectFileDetailGenericView(RetrieveDestroyAPIView):
    serializer_class = ProjectFileDetailSerializer

    def get_object(self):
        return get_object_or_404(ProjectFile, pk=self.kwargs['pk'])

    def retrieve(self, request: Request, *args, **kwargs) -> Response:
        task = self.get_object()

        serializer = self.get_serializer(task)

        return Response(
            data=serializer.data,
            status=status.HTTP_200_OK
        )
```

1

```python
def destroy(self, request: Request, *args, **kwargs) -> Response:
    task = self.get_object()

    try:
        delete_file(file_path=task.file_path.path)
    except Exception as e:
        return Response(
            data={"message": str(e)},
            status=status.HTTP_400_BAD_REQUEST
        )
    else:
        task.delete()
    return Response(
        data={
            "message": "File deleted successfully"
        },
        status=status.HTTP_200_OK
    )
```

2

Задание 2

Решение

3. Добавление функции на удаление файла (apps/projects/utils/upload_file_helpers.py):

```
def delete_file(file_path):
   os.remove(os.path.realpath(file_path))
```

Задание 2

Решение

4. Регистрация нового эндпоинта (apps/projects/urls.py):

```
from apps.projects.views.project_file_views import (
   ProjectFileListGenericView,
   ProjectFileDetailGenericView,
)


urlpatterns = [
   path('', ProjectListAPIView.as_view()),
   path('<int:pk>/', ProjectDetailAPIView.as_view()),
   path('files/', ProjectFileListGenericView.as_view()),
   path('files/<int:pk>/', ProjectFileDetailGenericView.as_view()),  # NEW
]
```

Задание 2

Решение

5. Запуск сервера, проверка **GET, DELETE** запросов.

6. Git команды (**git add, git commit, git push origin <branch_name>**)

Задание 3

Создание кастомного пользователя. Часть 1

1. Удалите существующую базу.
2. Создайте новое приложение **users**, зарегистрируйте его в проекте.
3. Создайте свою собственную модель **User**, в которой будут поля:
    - **username (**Строковое поле, не более 50 символов, уникальное, обязательно к заполнению**)**
    - **first_name (**строковое поле, не более 40 символов, обязательно к заполнению**)**
    - **last_name (**строковое поле, не более 40 символов, обязательно к заполнению**)**
    - **email (**поле email, уникальное, обязательное к заполнению)
    - **phone** (строковое поле, не более 75 символов, не обязательно к заполнению**)**
    - **is_staff (**административное логическое поле, по умолчанию False**)**
    - **is_active (**логическое поле, по умолчанию True**)**
    - **date_joined (**Поле даты и времени, заполняется автоматически при создании**)**
    - **last_login (**Поле даты и времени, заполняется при входе в систему**)**
    - **updated_at (**Поле даты и времени, заполняется автоматически при всех обновлениях**)**
    - **deleted_at (**Поле даты и времени, заполняется только если поле deleted переходит в состояние True**)**
    - **deleted (**Логическое поле, по умолчанию - True**)**
    - **project (**Связь с моделью **Project**. При удалении проекта все сотрудники так же удаляются, связующее поле у проекта **- users,** при создании пользователя проекта у него может и не быть**)**

Задание 3

Создание кастомного пользователя. Часть 2

- **position (**строковое поле выбора из готовых вариантов позиций:
    1. **Ceo**
    2. **Cto**
    3. **Designer**
    4. **Programmer**
    5. **Product_owner**
    6. **Project_owner**
    7. **Project_manager**
    8. **Qa)**

Задание 3

Создание кастомного пользователя. Часть 3

4. Укажите поле **email**, как поле для входа в систему.
5. Добавьте дополнительные поля для регистрации:
    - **username**
    - **last_name**
    - **first_name**

6. Перепишите настройки django на работу с пользователем в файле **settings.py**
7. Переопределите связь с новым пользователем в других моделях, где была связь с пользователем (модель **Task**).
8. Проведите миграции и убедитесь, что всё работает.

Задание 3

Решение

1. Создание нового проекта:

```
python manage.py startapp users
```

Задание 3

Решение

2. Создание своего пользователя (apps/users/models.py):

```python
from django.contrib.auth.base_user import AbstractBaseUser
from django.contrib.auth.models import PermissionsMixin, UserManager
from django.core.validators import MinLengthValidator
from django.db import models
from django.utils.translation import gettext_lazy as _

from apps.projects.models import Project
from apps.users.choices.positions import UserPositions

class User(AbstractBaseUser, PermissionsMixin):
   username = models.CharField(
       _("username"),
       max_length=50,
       unique=True,
       error_messages={
           "unique": _("A user with that username already exists."),
       }
   )
```

1

```python
first_name = models.CharField(
    _("first name"),
    max_length=40,
    validators=[MinLengthValidator(2)],
)
last_name = models.CharField(
    _("last name"),
    max_length=40,
    validators=[MinLengthValidator(2)],
)
email = models.EmailField(
    _("email address"),
    max_length=150,
    unique=True
)
phone = models.CharField(max_length=75, null=True, blank=True)
is_staff = models.BooleanField(default=False)
is_active = models.BooleanField(default=True)
```

2

Задание 3

Решение

```
    date_joined = models.DateTimeField(
        name="registered", auto_now_add=True
    )
    last_login = models.DateTimeField(null=True, blank=True)
    updated_at = models.DateTimeField(auto_now=True)
    deleted_at = models.DateTimeField(null=True, blank=True)
    deleted = models.BooleanField(default=False)
    position = models.CharField(
        max_length=15,
        choices=UserPositions.choices,
        default=UserPositions.PROGRAMMER
    )
    project = models.ForeignKey(
        Project,
        on_delete=models.CASCADE,
        related_name="users",
        null=True,
        blank=True,
)
```

3

```
USERNAME_FIELD = "email"
REQUIRED_FIELDS = [
    "username",
    "first_name",
    "last_name",
    “position”
 ]


objects = UserManager()

def __str__(self):
    return f"{self.last_name} {self.first_name}"
```

4

Задание 3

Решение

3. Создание класса с позициями (apps/users/choices/positions.py):

```
from enum import Enum


class Positions(Enum):
   CEO = "CEO"
   CTO = "CTO"
   DESIGNER = "Designer"
   PRODUCT_OWNER = "Product Owner"
   PROJECT_OWNER = "Project Owner"
   PROGRAMMER = "Programmer"
   PROJECT_MANAGER = "Project Manager"
   QA = "QA"

   @classmethod
   def choices(cls):
       return [(attr.name, attr.value) for attr in cls]
```

Задание 3

Решение

4. Обновление настроек проекта:

```
AUTH_USER_MODEL = 'users.User'
```

Задание 3

Решение

5. Создание и применение миграций:

```
python manage.py makemigrations
python manage.py migrate
```

6. Проверка, что всё хорошо, создание суперпользователя:

```
python manage.py createsuperuser
```

Задание 4

Получение списка пользователей, фильтрация пользователей

1. Создайте сериализатор **UserListSerialzier** для отображения только лишь некоторых полей для пользователя:
    - **first_name**
    - **last_name**
    - **position**
    - **email**
    - **phone**
    - **last_login**
2. Создайте классовое отображение **UserListGenericView** для получения списка пользователей.
    - Переопределите метод **get_queryset** для получения списка пользователей по конкретному проекту, если фильтр был передан, иначе должен отдаваться список всех пользователей.
3. Зарегистрируйте новый эндпоинт, протестируйте его, чтобы убедиться, что он работает.
4. Закомментируйте все изменения, создайте запрос на слияние.

Задание 4

Решение

1. Создание сериализатора (apps/users/serializers.py):

```
from rest_framework import serializers

from apps.users.models import User


class UserListSerializer(serializers.ModelSerializer):
   class Meta:
       model = User
       fields = (
           'first_name',
           'last_name',
           'position',
           'email',
           'phone',
           'last_login',
       )
```

Задание 4

Решение

2. Создание класса-отображения (apps/users/views.py):

```python
from rest_framework.request import Request
from rest_framework.response import Response
from rest_framework import status
from rest_framework.generics import ListAPIView

from apps.users.models import User
from apps.users.serializers import UserListSerializer


class UserListGenericView(ListAPIView):
   serializer_class = UserListSerializer
```

1

```python
def get_queryset(self):
    project_name = self.request.query_params.get('project_name')
    if project_name:
        return User.objects.filter(project__name=project_name)
    return User.objects.all()

def list(self, request: Request, *args, **kwargs) -> Response:
    projects = self.get_queryset()

    if not projects.exists():
        return Response(
            data=[],
            status=status.HTTP_204_NO_CONTENT
        )
    serializer = self.get_serializer(projects, many=True)
    return Response(
        serializer.data,
        status=status.HTTP_200_OK
    )
```

2

Задание 4

Решение

3. Регистрация нового списка эндпоинтов (apps/router.py):

```
from django.urls import path, include


urlpatterns = [
   path('tasks/', include('apps.tasks.urls')),
   path('projects/', include('apps.projects.urls')),
   path('users/', include('apps.users.urls'))  # NEW
]
```

Задание 4

Решение

4. Регистрация нового эндпоинта (apps/users/urls.py):

```
from django.urls import path

from apps.users.views import UserListGenericView


urlpatterns = [
   path('', UserListGenericView.as_view()),
]
```

Задание 4

Решение

5. Запустите сервер, проверьте **GET** запрос.

6. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 5

Регистрация нового пользователя. Часть 1

1. Создайте новый сериализатор **RegisterUserSerializer** для регистрации нового пользователя. Сериализатор будет обрабатывать поля:
   - **username**
   - **first_name**
   - **last_name**
   - **email**
   - **position**
   - **password**
   - **re_password**

   Поле **re_password** - искусственно созданное поле для проверки совпадения паролей, содержит настройки:
   - Максимальная длина - 128 символов
   - Только для записи

   В параметре **extra_kwargs** указать, что поле **password** строго только для записи.

Задание 5

Регистрация нового пользователя. Часть 2

2. Создайте в этом сериализаторе метод **validate**. Этот метод должен проверять:
    - **username** (Любые буквы из латиницы, символ нижнего подчёркивания, цифры от 0 до 9)
    - **first_name** (Строго буквы из латиницы, любым регистром)
    - **last_name** (Строго буквы из латиницы, любым регистром)
    - **password** (пароль и его повторный ввод должны совпадать, так же вызов базовых проверок пароля от системы **Django**)
3. Переопределите метод **create** для хеширвания пароля и его установке для пользователя перед созданием.
4. Напишите классовое отображение, которое будет принимать данные из запроса и создавать пользователя.
5. Зарегистрируйте новый эндпоинт, протестируйте его, чтобы убедиться, что он работает.
6. Закомментируйте все изменения, создайте запрос на слияние.

Задание 5

Решение

1. Создание сериализатора (apps/users/serializers.py):

```python
from django.core.exceptions import ValidationError
from django.contrib.auth.password_validation import
validate_password

import re


class RegisterUserSerializer(serializers.ModelSerializer):
   re_password = serializers.CharField(
       max_length=128,
       write_only=True,
   )
```

1

```python
class Meta:
    model = User
    fields = (
        'username',
        'first_name',
        'last_name',
        'email',
        'position',
        'password',
        're_password',
    )
    extra_kwargs = {
        'password': {'write_only': True}
    }

def validate(self, data):
    username = data.get('username')
    first_name = data.get('first_name')
    last_name = data.get('last_name')
```

2

Задание 5

Решение

```
        if not re.match('^[a-zA-Z0-9_]*$', username):
            raise serializers.ValidationError(
                "The username must be alphanumeric characters or
have only _ symbol"
            )
        if not re.match('^[a-zA-Z]*$', first_name):
            raise serializers.ValidationError(
                "The first name must contain only alphabet
symbols"
            )
        if not re.match('^[a-zA-Z]*$', last_name):
            raise serializers.ValidationError(
                "The last name must contain only alphabet symbols"
            )

        password = data.get("password")
        re_password = data.get("re_password")

        if password != re_password:
            raise serializers.ValidationError({"password":
"Passwords don't match"})
```

3

```
        try:
            validate_password(password)
        except ValidationError as err:
            raise serializers.ValidationError({"password":
err.messages})

        return data

    def create(self, validated_data):
        password = validated_data.pop('password')
        validated_data.pop('re_password')
        user = User(**validated_data)
        user.set_password(password)
        user.save()
        return user
```

4

Задание 5

Решение

2. Создание классового отображения (apps/users/views.py):

```
class RegisterUserGenericView(CreateAPIView):
   serializer_class = RegisterUserSerializer


   def create(self, request: Request, *args, **kwargs) -> Response:
       serializer = self.get_serializer(data=request.data)
       serializer.is_valid(raise_exception=True)
       serializer.save()
       headers = self.get_success_headers(serializer.data)
       return Response(serializer.data, status=status.HTTP_201_CREATED, headers=headers)
```

Задание 5

Решение

3. Регистрация нового эндпоинта (apps/users/urls.py):

```
from apps.users.views import (
   UserListGenericView,
   RegisterUserGenericView
)

urlpatterns = [
   path('', UserListGenericView.as_view()),
   path('register/', RegisterUserGenericView.as_view()),
]
```

Задание 5

Решение

4. Запуск сервера, проверка **GET, DELETE** запросов.

5. Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 6

Обновление сериализатора для создания задач

1. В сериализатор **CreateUpdateTaskSerializer** добавьте новое поле **assignee** для возможности указывать назначенного разработчика на определённую задачу при создании\обновлении задачи.
    - Поле заполняется по **email** пользователя
    - Поле не обязательно к заполнению, его можно пропускать
2. Запустите сервер и протестируйте добавленные изменения, чтобы убедиться, что создание и обновление задач работает.
3. Закомментируйте все изменения, создайте запрос на слияние.

Задание 6

Решение

1. Добавление нового поля (apps/tasks/serializers/task_serializers.py):

```
class CreateUpdateTaskSerializer(serializers.ModelSerializer):
    ...
    assignee = serializers.SlugRelatedField(  # NEW
        slug_field='email',
        queryset=User.objects.all(),
        required=False
    )

    class Meta:
        model = Task
        fields = (
            'deadline',
            'Assignee'  # NEW
        )
```

Задание 6

Решение

2. Запуск сервера, проверка **POST, PUT** запросов.

3. Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 7

Скачивание конкретного файла по переданному ID

1. Напишите отображение для возможности скачивания файла по переданному **ID** файла. Если файла с таким **ID** не существует - выдавать сообщение, что файл не найден.
2. Зарегистрируйте новое отображение, запустите сервер и протестируйте новый функционал по скачиванию файлов.
3. Закомментируйте все изменения, создайте запрос на слияние.

Задание 7

Решение

1. Создание отображения (apps/projects/views/project_file_views.py):

```
class DownloadProjectFileView(APIView):
    def get_object(self):
        return get_object_or_404(ProjectFile, pk=self.kwargs['pk'])

    def get(self, request: Request, *args, **kwargs) -> FileResponse:
        project_file = self.get_object()

        file_handle = project_file.file_path.open()

        response = FileResponse(file_handle, content_type='application/octet-stream')
        response['Content-Disposition'] = f'attachment; filename="{project_file.file_name}"'

        return response
```

Задание 7

Решение

2. Регистрация нового ендпоинта (apps/projects/urls.py):

```
from apps.projects.views.project_file_views import (
   ProjectFileListGenericView,
   ProjectFileDetailGenericView,
   DownloadProjectFileView  # NEW
)

urlpatterns = [
   path('', ProjectsListAPIView.as_view()),
   path('<int:pk>/', ProjectDetailAPIView.as_view()),
   path('files/', ProjectFileListGenericView.as_view()),
   path('files/download/<int:pk>/', DownloadProjectFileView.as_view()),  # NEW
   path('files/<int:pk>/', ProjectFileDetailGenericView.as_view()),
]
```

Задание 7

Решение

3. Запуск сервера, проверка **POST, PUT** запросов.

4. Git команды (**git add, git commit, git push origin <branch_name>**).

IT CAREER HUB

ВОПРОСЫ

Заключение

